
Report
Implementation and Study of Scalable Big Data Processing and Analytics

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Aayush Ranjan (2210991137)

Shubham Thakur (2210992370)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

***CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND
TECHNOLOGY CHITKARA UNIVERSITY, PUNJAB, INDIA***

Sr. No.	Topic	Page No.
1.	Abstract	3
2.	Introduction	4
3.	Objective	4
4.	Methodology	5-6
5.	Results	6-7
6.	Discussion	7-8
7.	Data Quality Observations	8
8.	Limitations	9
9.	Conclusion	9
10	Project Implementation Overview	10
11.	References	10

1. Abstract

The rapid growth of data in domains such as business analytics, IoT, and e-commerce has made the selection of efficient ETL (Extract–Transform–Load) frameworks critical. This report presents a comparative study of Pandas (v2.x) and Apache PySpark (v3.x) using a controlled benchmarking environment.

Experiments were conducted on datasets ranging from 10,000 to 10 million rows using a four-stage ETL pipeline. Results show that PySpark is significantly faster in data extraction (up to 7.2× at large scale), while Pandas performs better in overall execution time for datasets up to 10 million rows. However, PySpark demonstrates superior memory efficiency, using up to 89.6% less memory compared to Pandas.

The study provides practical insights into when to use each framework based on data size, memory constraints, and system requirements.

2. Introduction

Modern data-driven systems rely heavily on efficient data pipelines. Raw data from sources such as transaction logs, sensors, and social media must be processed before meaningful insights can be extracted.

Pandas is widely used due to its simplicity and efficiency for small datasets. However, it struggles when data exceeds available memory.

PySpark, on the other hand, uses distributed processing to handle large-scale data efficiently. While powerful, it introduces overhead such as task scheduling and data shuffling.

This report aims to answer:

- At what data size does PySpark become beneficial?
 - How do different ETL stages affect performance?
 - How does memory usage impact real-world deployment?
-

3. Objectives

- Compare Pandas and PySpark across ETL stages
 - Analyze performance at different data scales
 - Evaluate memory usage patterns
 - Identify optimal use cases for each framework
-

4. Methodology

4.1 Experimental Setup

Parameter	Value
OS	Ubuntu 22.04
CPU	12-core
Python	3.10
Pandas	2.x
PySpark	3.4 (local[12])
Memory Tracking	psutil
Output Format	Parquet

4.2 Dataset Design

- Synthetic dataset generated using Faker
- Includes:
 - Transaction ID, Customer ID
 - Price, Quantity, Discount
 - Region, Category, Payment Method
 - Review Score (with 0.14% missing values)

Dataset Sizes:

- 10K
 - 100K
 - 1M
 - 5M
 - 10M rows
-

4.3 ETL Pipeline

Both frameworks followed identical stages:

1. **Extract** – Read CSV data
 2. **Transform** – Clean data, handle nulls, create revenue column
 3. **Load** – Save as Parquet
 4. **Analytics** – Generate KPIs (category revenue, trends, top reps)
-

5. Results

5.1 Total Execution Time

Scale Pandas (s) PySpark (s)

10K	0.254	19.681
100K	0.595	9.828
1M	5.928	22.323
10M	102.001	124.396

Key Insight:

- Pandas is faster for small and medium datasets
 - PySpark overhead reduces as data size increases
-

5.2 Stage-wise Analysis

Extract Stage

- PySpark: **7.2× faster**
- Reason: Parallel file reading

Transform Stage

- PySpark: **~62% slower**

- Reason: Shuffle operations & JVM overhead

Load Stage

- Slightly slower in PySpark due to distributed writes

Analytics Stage

- Pandas much faster due to in-memory vectorized operations

5.3 Memory Usage

Scale Pandas (MB) PySpark (MB)

100K	172.7	189.9
1M	282.8	187.1
10M	1400.9	145.6

Key Insight:

- PySpark uses **~90% less memory at large scale**
- Pandas memory grows linearly with data

6. Discussion

6.1 Crossover Point

- PySpark becomes competitive around **15M–25M rows**
- But memory efficiency makes it useful earlier

6.2 Transform Bottleneck in PySpark

Major issues:

- Data shuffling
- Python ↔ JVM communication
- Row-by-row operations

Optimization:

- Use vectorized UDFs (Pandas UDFs)
 - Cache intermediate data
-

6.3 When to Use Which Tool

Use Pandas when:

- Data fits in memory
- Fast development needed
- Single-machine processing

Use PySpark when:

- Data is very large
 - Memory is limited
 - Distributed systems are required
-

7. Data Quality Observations

- Pandas: Flexible but delayed validation
- PySpark: Strict schema enforcement

PySpark is better for **data governance and auditing**

8. Limitations

- Single-machine testing (no cluster)
 - Synthetic dataset
 - No failure recovery testing
 - Other tools (Dask, Polars) not included
-

9. Conclusion

This study shows that:

- Pandas is faster for smaller datasets
- PySpark scales better for large datasets
- Memory efficiency is PySpark's biggest advantage

Final Takeaway:

- **Performance $\neq$ Just Time**
 - **Memory + Scalability = Real-world performance**
-

10. Project Implementation Overview

Based on your project structure :

- `pandas_pipeline.py` → Single-node ETL
 - `pyspark_pipeline.py` → Distributed ETL
 - `benchmark.py` → Performance comparison
 - `visualize_results.py` → Charts & dashboard
-

11. References

- T. Hey, S. Tansley and K. Tolle, The Fourth Paradigm: Data-Intensive Scientific Discovery. Redmond, WA: Microsoft Research, 2009.
- R. Kimball and M. Ross The Data Warehouse Toolkit, 3rd edition. Indianapolis, IN: Wiley, 2013.
- W. McKinney says data structures are very important for computing in Python. He talked about it in Proc. 9Th Python in Science Conf. (SciPy) Austin, TX, 2010 pages 56–61.
- S. Owen, R. Anil, T. Dunning and E. Friedman wrote a book called Mahout in Action. It was published by Manning Publications in Greenwich, CT in 2011.
- M. Armbrust and others wrote about Spark SQL. They talked about data processing in Spark at the ACM SIGMOD Int. Conf. Management of Data in 2015. It was on pages 1383–1394.